



Trabalho Pratico

Implementacao de cadastro de clientes em modo texto, com persistencia em arquivos binarios, utilizando Java.

Universidade	Estacio de Sa
Campus/Polo	POLO TOM JOBIM - BARRA DA TIJUCA - RJ
Curso contratado	DESENVOLVIMENTO FULL STACK
Disciplina	Programacao Back-end Com Java
Periodo letivo	2026.1
Integrante	Clayton Luis Meireles Goncalves
Turma	9001
Repositorio Git	Repositorio local: C:\Users\clayton.goncalves\Documents\CadastroPOO
Link remoto	https://github.com/Cl4y7oon/CadastroPOO

Titulo da pratica

Implementacao de um cadastro de clientes em modo texto com persistencia em arquivos, baseado na tecnologia Java.

Objetivo da pratica

Desenvolver um sistema cadastral em Java aplicando programacao orientada a objetos, heranca, polimorfismo, controle de excecoes, repositórios em memoria e persistencia de objetos em arquivos binarios.

1. Descricao da Implementacao

- O projeto foi criado com o nome CadastroPOO, seguindo a estrutura de uma Java Application do NetBeans com Ant.
- As entidades foram organizadas no pacote model.
- A classe Pessoa possui id, nome, construtor padrao, construtor completo, getters, setters e metodo exibir.
- As classes PessoaFisica e PessoaJuridica herdam de Pessoa e sobrescrevem o metodo exibir.
- Todas as entidades implementam Serializable.
- Os repositórios mantem ArrayList privado e oferecem inserir, alterar, excluir, obter, obterTodos, persistir e recuperar.
- A classe principal implementa o cadastro em modo texto com as opcoes solicitadas no roteiro.

2. Codigos Solicitados

src/model/Pessoa.java

```
package model;

import java.io.Serializable;

public class Pessoa implements Serializable {

    private static final long serialVersionUID = 1L;

    private int id;
    private String nome;

    public Pessoa() {
    }

    public Pessoa(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    public void exibir() {
        System.out.println("ID: " + id);
        System.out.println("Nome: " + nome);
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
}
```

src/model/PessoaFisica.java

```
package model;

import java.io.Serializable;

public class PessoaFisica extends Pessoa implements Serializable {

    private static final long serialVersionUID = 1L;

    private String cpf;
    private int idade;

    public PessoaFisica() {
    }

    public PessoaFisica(int id, String nome, String cpf, int idade) {
        super(id, nome);
    }
}
```

```

        this.cpf = cpf;
        this.idade = idade;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CPF: " + cpf);
        System.out.println("Idade: " + idade);
    }

    public String getCpf() {
        return cpf;
    }

    public void setCpf(String cpf) {
        this.cpf = cpf;
    }

    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }
}

```

src/model/PessoaJuridica.java

```

package model;

import java.io.Serializable;

public class PessoaJuridica extends Pessoa implements Serializable {

    private static final long serialVersionUID = 1L;

    private String cnpj;

    public PessoaJuridica() {
    }

    public PessoaJuridica(int id, String nome, String cnpj) {
        super(id, nome);
        this.cnpj = cnpj;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CNPJ: " + cnpj);
    }

    public String getCnpj() {
        return cnpj;
    }

    public void setCnpj(String cnpj) {
        this.cnpj = cnpj;
    }
}

```

src/model/PessoaFisicaRepo.java

```

package model;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;

public class PessoaFisicaRepo {

    private ArrayList<PessoaFisica> pessoasFisicas;

    public PessoaFisicaRepo() {
        pessoasFisicas = new ArrayList<>();
    }

    public void inserir(PessoaFisica pessoaFisica) {
        pessoasFisicas.add(pessoaFisica);
    }

    public void alterar(PessoaFisica pessoaFisica) {
        PessoaFisica pessoaAtual = obter(pessoaFisica.getId());
        if (pessoaAtual != null) {

```

```

        int indice = pessoasFisicas.indexOf(pessoaAtual);
        pessoasFisicas.set(indice, pessoaFisica);
    }
}

public void excluir(int id) {
    PessoaFisica pessoaFisica = obter(id);
    if (pessoaFisica != null) {
        pessoasFisicas.remove(pessoaFisica);
    }
}

public PessoaFisica obter(int id) {
    for (PessoaFisica pessoaFisica : pessoasFisicas) {
        if (pessoaFisica.getId() == id) {
            return pessoaFisica;
        }
    }
    return null;
}

public ArrayList<PessoaFisica> obterTodos() {
    return pessoasFisicas;
}

public void persistir(String nomeArquivo) throws IOException {
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(nomeArquivo))) {
        out.writeObject(pessoasFisicas);
    }
}

@SuppressWarnings("unchecked")
public void recuperar(String nomeArquivo) throws IOException, ClassNotFoundException {
    try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(nomeArquivo))) {
        pessoasFisicas = (ArrayList<PessoaFisica>) in.readObject();
    }
}
}

```

src/model/PessoaJuridicaRepo.java

```

package model;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;

public class PessoaJuridicaRepo {

    private ArrayList<PessoaJuridica> pessoasJuridicas;

    public PessoaJuridicaRepo() {
        pessoasJuridicas = new ArrayList<>();
    }

    public void inserir(PessoaJuridica pessoaJuridica) {
        pessoasJuridicas.add(pessoaJuridica);
    }

    public void alterar(PessoaJuridica pessoaJuridica) {
        PessoaJuridica pessoaAtual = obter(pessoaJuridica.getId());
        if (pessoaAtual != null) {
            int indice = pessoasJuridicas.indexOf(pessoaAtual);
            pessoasJuridicas.set(indice, pessoaJuridica);
        }
    }

    public void excluir(int id) {
        PessoaJuridica pessoaJuridica = obter(id);
        if (pessoaJuridica != null) {
            pessoasJuridicas.remove(pessoaJuridica);
        }
    }

    public PessoaJuridica obter(int id) {
        for (PessoaJuridica pessoaJuridica : pessoasJuridicas) {
            if (pessoaJuridica.getId() == id) {
                return pessoaJuridica;
            }
        }
        return null;
    }

    public ArrayList<PessoaJuridica> obterTodos() {
        return pessoasJuridicas;
    }

    public void persistir(String nomeArquivo) throws IOException {

```

```

        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(nomeArquivo))) {
            out.writeObject(pessoasJuridicas);
        }
    }

    @SuppressWarnings("unchecked")
    public void recuperar(String nomeArquivo) throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(nomeArquivo))) {
            pessoasJuridicas = (ArrayList<PessoaJuridica>) in.readObject();
        }
    }
}

```

src/cadastropoo/CadastroPOO.java

```

package cadastropoo;

import java.util.Scanner;
import model.PessoaFisica;
import model.PessoaFisicaRepo;
import model.PessoaJuridica;
import model.PessoaJuridicaRepo;

public class CadastroPOO {

    private static final Scanner ENTRADA = new Scanner(System.in);
    private static final PessoaFisicaRepo REPO_FISICA = new PessoaFisicaRepo();
    private static final PessoaJuridicaRepo REPO_JURIDICA = new PessoaJuridicaRepo();

    public static void main(String[] args) {
        int opcao;

        do {
            exibirMenu();
            opcao = lerInteiro("Opcao: ");

            switch (opcao) {
                case 1:
                    incluir();
                    break;
                case 2:
                    alterar();
                    break;
                case 3:
                    excluir();
                    break;
                case 4:
                    exibirPorId();
                    break;
                case 5:
                    exibirTodos();
                    break;
                case 6:
                    salvar();
                    break;
                case 7:
                    recuperar();
                    break;
                case 0:
                    System.out.println("Sistema finalizado.");
                    break;
                default:
                    System.out.println("Opcao invalida.");
                    break;
            }
        } while (opcao != 0);
    }

    private static void exibirMenu() {
        System.out.println();
        System.out.println("1 - Incluir");
        System.out.println("2 - Alterar");
        System.out.println("3 - Excluir");
        System.out.println("4 - Exibir pelo id");
        System.out.println("5 - Exibir todos");
        System.out.println("6 - Salvar dados");
        System.out.println("7 - Recuperar dados");
        System.out.println("0 - Finalizar");
    }

    private static void incluir() {
        String tipo = lerTipo();
        if (tipo.equalsIgnoreCase("F")) {
            REPO_FISICA.inserir(lerPessoaFisica());
        } else {
            REPO_JURIDICA.inserir(lerPessoaJuridica());
        }
    }

    private static void alterar() {

```

```

String tipo = lerTipo();
int id = lerInteiro("ID: ");

if (tipo.equalsIgnoreCase("F")) {
    PessoaFisica pessoaFisica = REPO_FISICA.obter(id);
    if (pessoaFisica != null) {
        pessoaFisica.exibir();
        REPO_FISICA.alterar(lerPessoaFisicaComId(id));
    } else {
        System.out.println("Pessoa fisica nao encontrada.");
    }
} else {
    PessoaJuridica pessoaJuridica = REPO_JURIDICA.obter(id);
    if (pessoaJuridica != null) {
        pessoaJuridica.exibir();
        REPO_JURIDICA.alterar(lerPessoaJuridicaComId(id));
    } else {
        System.out.println("Pessoa juridica nao encontrada.");
    }
}
}

private static void excluir() {
    String tipo = lerTipo();
    int id = lerInteiro("ID: ");

    if (tipo.equalsIgnoreCase("F")) {
        REPO_FISICA.excluir(id);
    } else {
        REPO_JURIDICA.excluir(id);
    }
}

private static void exibirPorId() {
    String tipo = lerTipo();
    int id = lerInteiro("ID: ");

    if (tipo.equalsIgnoreCase("F")) {
        PessoaFisica pessoaFisica = REPO_FISICA.obter(id);
        if (pessoaFisica != null) {
            pessoaFisica.exibir();
        } else {
            System.out.println("Pessoa fisica nao encontrada.");
        }
    } else {
        PessoaJuridica pessoaJuridica = REPO_JURIDICA.obter(id);
        if (pessoaJuridica != null) {
            pessoaJuridica.exibir();
        } else {
            System.out.println("Pessoa juridica nao encontrada.");
        }
    }
}

private static void exibirTodos() {
    String tipo = lerTipo();

    if (tipo.equalsIgnoreCase("F")) {
        for (PessoaFisica pessoaFisica : REPO_FISICA.obterTodos()) {
            pessoaFisica.exibir();
            System.out.println();
        }
    } else {
        for (PessoaJuridica pessoaJuridica : REPO_JURIDICA.obterTodos()) {
            pessoaJuridica.exibir();
            System.out.println();
        }
    }
}

private static void salvar() {
    System.out.print("Prefixo dos arquivos: ");
    String prefixo = ENTRADA.nextLine();

    try {
        REPO_FISICA.persistir(prefixo + ".fisica.bin");
        REPO_JURIDICA.persistir(prefixo + ".juridica.bin");
        System.out.println("Dados salvos com sucesso.");
    } catch (Exception e) {
        System.out.println("Erro ao salvar dados: " + e.getMessage());
    }
}

private static void recuperar() {
    System.out.print("Prefixo dos arquivos: ");
    String prefixo = ENTRADA.nextLine();

    try {
        REPO_FISICA.recuperar(prefixo + ".fisica.bin");
        REPO_JURIDICA.recuperar(prefixo + ".juridica.bin");
    }
}

```

```

        System.out.println("Dados recuperados com sucesso.");
    } catch (Exception e) {
        System.out.println("Erro ao recuperar dados: " + e.getMessage());
    }
}

private static PessoaFisica lerPessoaFisica() {
    int id = lerInteiro("ID: ");
    return lerPessoaFisicaComId(id);
}

private static PessoaFisica lerPessoaFisicaComId(int id) {
    System.out.print("Nome: ");
    String nome = ENTRADA.nextLine();
    System.out.print("CPF: ");
    String cpf = ENTRADA.nextLine();
    int idade = lerInteiro("Idade: ");

    return new PessoaFisica(id, nome, cpf, idade);
}

private static PessoaJuridica lerPessoaJuridica() {
    int id = lerInteiro("ID: ");
    return lerPessoaJuridicaComId(id);
}

private static PessoaJuridica lerPessoaJuridicaComId(int id) {
    System.out.print("Nome: ");
    String nome = ENTRADA.nextLine();
    System.out.print("CNPJ: ");
    String cnpj = ENTRADA.nextLine();

    return new PessoaJuridica(id, nome, cnpj);
}

private static String lerTipo() {
    String tipo;
    do {
        System.out.print("Tipo (F - Fisica / J - Juridica): ");
        tipo = ENTRADA.nextLine();
    } while (!tipo.equalsIgnoreCase("F") && !tipo.equalsIgnoreCase("J"));

    return tipo;
}

private static int lerInteiro(String mensagem) {
    System.out.print(mensagem);
    int valor = ENTRADA.nextInt();
    ENTRADA.nextLine();
    return valor;
}
}

```

3. Resultado da Execucao

A execucao abaixo demonstra inclusao de pessoa fisica, inclusao de pessoa juridica, exibicao por id, exibicao de todos os registros, salvamento em arquivos binarios, recuperacao dos dados e finalizacao do sistema.

```
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo id
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Finalizar
Opcao: Tipo (F - Fisica / J - Juridica): ID: Nome: CPF: Idade:
```

```
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo id
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Finalizar
Opcao: Tipo (F - Fisica / J - Juridica): ID: Nome: CNPJ:
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo id
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Finalizar
```

```
Opcao: Tipo (F - Fisica / J - Juridica): ID: ID: 1
Nome: Maria Silva
CPF: 11122233344
Idade: 30
```

```
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo id
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Finalizar
Opcao: Tipo (F - Fisica / J - Juridica): ID: 1
Nome: Maria Silva
CPF: 11122233344
Idade: 30
```

```
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo id
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Finalizar
Opcao: Tipo (F - Fisica / J - Juridica): ID: 2
Nome: Empresa Teste
CNPJ: 11222333000144
```

```
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo id
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Finalizar
Opcao: Prefixo dos arquivos: Dados salvos com sucesso.
```

```
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo id
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Finalizar
Opcao: Prefixo dos arquivos: Dados recuperados com sucesso.
```

```
1 - Incluir
2 - Alterar
```

3 - Excluir
4 - Exibir pelo id
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Finalizar
Opcao: Sistema finalizado.

Arquivos gerados durante o teste: documentacao_teste.fisica.bin e documentacao_teste.juridica.bin.

4. Analise e Conclusao

Quais as vantagens e desvantagens do uso de heranca?

A heranca permite reaproveitar atributos e comportamentos comuns, como id, nome e exibir, evitando duplicacao entre pessoa fisica e pessoa juridica. Tambem facilita o polimorfismo, pois subclasses podem especializar o comportamento herdado. A desvantagem e que o acoplamento entre classe pai e filhas aumenta; mudancas na superclasse podem afetar todas as subclasses, e uma hierarquia mal planejada pode deixar o sistema rigido.

Por que a interface `Serializable` e necessaria ao efetuar persistencia em arquivos binarios?

`Serializable` indica que os objetos podem ser transformados em um fluxo de bytes pela JVM. Como os repositorios usam `ObjectOutputStream` e `ObjectInputStream`, as classes persistidas precisam implementar essa interface para que seus objetos possam ser gravados e reconstruidos a partir dos arquivos binarios.

Como o paradigma funcional e utilizado pela API stream no Java?

A API Stream utiliza conceitos do paradigma funcional ao permitir operacoes declarativas sobre colecoes, como filter, map, sorted, reduce e forEach. Essas operacoes recebem funcoes ou expressoes lambda, reduzindo a necessidade de controlar manualmente lacos e estados intermediarios.

Quando trabalhamos com Java, qual padrao de desenvolvimento e adotado na persistencia de dados em arquivos?

Neste trabalho foi adotado o padrao de repositorio, separando a logica de acesso e persistencia dos dados das entidades e da interface de usuario. Os repositorios concentram as operacoes de cadastro e a gravacao/recuperacao em arquivos binarios.

O que sao elementos estaticos e qual o motivo para o metodo main adotar esse modificador?

Elementos estaticos pertencem a classe, e nao a uma instancia especifica. O metodo main e estatico porque a JVM precisa inicia-lo diretamente pela classe principal, antes de existir qualquer objeto criado pela aplicacao.

Para que serve a classe `Scanner`?

`Scanner` serve para ler dados de uma fonte de entrada, como o teclado. Neste sistema, ela e usada para receber opcoes do menu, identificadores, nomes, CPF, CNPJ, idade e prefixos dos arquivos informados pelo usuario.

Como o uso de classes de repositorio impactou na organizacao do codigo?

As classes de repositorio melhoraram a organizacao ao separar responsabilidades. As entidades representam dados e comportamento basico, a classe principal cuida da interacao em modo texto, e os repositorios centralizam inclusao, alteracao, exclusao, consulta e persistencia.

Conclusao Final

O trabalho permitiu aplicar os principais recursos de programacao orientada a objetos em Java em um sistema simples e funcional. A implementacao demonstrou o uso de heranca, polimorfismo, encapsulamento, tratamento de excecoes e persistencia binaria, alem de reforcar a importancia de organizar o codigo em

entidades, repositórios e camada de interação com o usuário.

Fonte do logo

Logo da Estácio obtido de Wikimedia Commons, arquivo ESTACIO-DE-SA-LOGO.png, com autoria indicada como Universidade Estácio de Sá.